# Taller Batch y Streaming

## Actividad 1

Para cada uno de los siguientes escenarios, indica:

- ¿Batch o Streaming? Justifica tu respuesta
- ¿Qué latencia sería aceptable?

Escenarios:

- Generar el reporte mensual de ventas.
- Detectar una transacción fraudulenta.
- Analizar el consumo eléctrico anual.
- Emitir una alerta cuando un servidor se cae.
- Calcular la nómina de los empleados.
- Mostrar en un dashboard cuántos usuarios están activos ahora.
- Analizar tendencias de compra del último año.
- Enviar una notificación cuando un sensor supere un umbral crítico.

## Actividad 2

*Una plataforma de comercio electrónico quiere* detectar fraude en tiempo real, mostrar métricas en vivo y generar reportes históricos para la gerencia.

Diseña una **arquitectura conceptual** que incluya:

- Componentes batch.
- Componentes de streaming.
- Qué datos se procesan en cada uno.
- Qué tipo de decisiones toma cada enfoque.

**Actividad 3**

Utilizaremos un **dataset público de eventos IoT**, que simula lecturas de sensores en el tiempo: ***IoT Sensor Data***

*Disponible en:*

https://www.kaggle.com/datasets/garystafford/environmental-sensor-data-132k

El dataset contiene lecturas de sensores ambientales y puede interpretarse como un **flujo de eventos históricos**.

Campos relevantes que usaremos:

- ts: instante en el que ocurrió el evento
- device: identificador del sensor
- temp: temperatura medida

Aunque el dataset está almacenado como archivo, **conceptualmente representa un stream de eventos**.

**Entorno de trabajo**

- Notebook en **Google Colab**
- Lenguaje: **Python**
- Librerías sugeridas: pandas, datetime

**Parte 1 – Preparación del dataset**

1. Descarga el dataset y cárgalo en un Notebook.
2. Convierte el campo ts a formato timestamp.
3. Ordena los eventos por tiempo ascendente.
4. Identifica: rango temporal del dataset, número total de eventos, número de sensores únicos

El campo ts está expresado en formato UNIX epoch time, es decir: número de segundos desde el 1 de enero de 1970 (UTC) almacenado como número decimal en notación científica.

Este formato es muy común en sistemas de streaming y plataformas de eventos, porque: es compacto, es fácil de ordenar, no depende de zona horaria.

**Conversión de ts a timestamp**

Antes de realizar cualquier análisis, debes convertir ts a un objeto fecha/hora.

Puedes basarte en el siguiente código para realizar la conversión

```
import pandas as pd

df = pd.read_csv("sensor_data.csv")

# Convertir ts (epoch en segundos) a timestamp UTC

df["timestamp"] = pd.to_datetime(df["ts"], unit="s", utc=True)
```

**Ordena los eventos por tiempo ascendente**

Verifica que la columna timestamp existe y es de tipo fecha/hora. Para ordenar los eventos por tiempo ascendente utiliza la siguiente instrucción:

```
df = df.sort_values("timestamp")
```

**Parte 2 – Procesamiento Batch**

Vamos a resolver preguntas analíticas **procesando el dataset completo como un lote**.

Asume que todos los datos ya fueron recolectados y que ahora se ejecuta un **job batch** para generar reportes históricos.

**Actividades**

Implementa en tu notebook:

1. Promedio diario de temperatura por sensor.
2. Valor máximo y mínimo de temperatura por sensor.
3. Número total de eventos por día.
4. Sensor con mayor número de lecturas en todo el período.
5. Día con mayor volumen de eventos.

**Parte 3 – Procesamiento tipo Streaming (simulado)**

Vamos a resolver un problema **reactivo**, simulando llegada de eventos en tiempo real. Ahora asume que los eventos llegan **uno a uno**, en orden temporal, y deben procesarse **a medida que ocurren**.

**Actividades streaming**

En tu notebook:

1. Itera sobre los eventos en orden temporal.
2. Implementa un contador incremental de eventos procesados.
3. Calcula un **promedio móvil de temperatura** por sensor usando una ventana temporal.
4. Genera una alerta (mensaje en consola) cuando la temperatura supere un umbral definido por ti.
5. Introduce un pequeño sleep() para simular llegada real.